Database Normalization Report

Inventory and Sales Management System

AASTU · Software Engineering · Database Systems · Group 10

This document demonstrates the normalization of the Inventory and Sales Management System database through **First Normal Form (1NF)**, **Second Normal Form (2NF)**, **Third Normal Form (3NF)**, and **Boyce-Codd Normal Form (BCNF)**. Each step removes data redundancy and update anomalies, producing a clean, dependency-preserving relational schema.

Normal Form	Key Rule	Status
1NF	Atomic values, no repeating groups	SATISFIED
2NF	No partial dependencies on composite PK	SATISFIED
3NF	No transitive dependencies	SATISFIED
BCNF	Every determinant is a candidate key	SATISFIED

Step 0 — Unnormalized Form (UNF)

Before normalization, all data lives in a single flat relation. This creates **repeating groups**, **update anomalies**, and **data redundancy**.

sale_id	sale_date	customer_name	customer_phone	product_name	supplier_name	price	qty_sold	qty_stock
1	2024-01-10	Alice	0911...	Widget A	SupplierX	25.00	2	50
2	2024-01-11	Bob	0922...	Widget A	SupplierX	25.00	1	48
3	2024-01-12	Alice	0911...	Gadget B	SupplierY	40.00	3	20

Problems: *customer_name and supplier_name repeat for every sale row. Updating a supplier name requires changing many rows. Deleting the last sale of a product loses the product data entirely.*

Step 1 — First Normal Form (1NF)

Rule: Each column must contain atomic (indivisible) values. No repeating groups or multi-valued attributes.

Actions taken:

1. Assigned a primary key to every relation.
2. Ensured every attribute holds a single value (e.g., phone is one number per row).
3. Eliminated any lists or comma-separated values.

Resulting relations after 1NF:

Sale_UNF (1NF base)
sale_id (PK)
product_id (FK)
customer_id (FK)
quantity_sold
sale_date

Product_UNF (1NF base)
product_id (PK)
supplier_id (FK)
name
price

Customer_UNF (1NF base)

customer_id (PK)
name
phone

Supplier_UNF (1NF base)
supplier_id (PK)
name
phone

All relations now contain only atomic values. 1NF is satisfied.

Step 2 — Second Normal Form (2NF)

Rule: Must be in 1NF, and every non-key attribute must be **fully functionally dependent** on the entire primary key (relevant when the PK is composite).

In this schema, **Sale** is the only relation that could have a composite key. All other tables already have single-column PKs, so they are trivially in 2NF.

Functional Dependencies identified:

Determinant	→	Dependent
`sale_id`	→	`product_id, customer_id, quantity_sold, sale_date`
`product_id`	→	`name, price, supplier_id`
`customer_id`	→	`name, phone`
`supplier_id`	→	`name, phone`

Since each relation uses a single-column primary key, there are **no partial dependencies**. Every non-key attribute depends on the full PK, not just part of it.

Schema after 2NF (unchanged from 1NF):

Relation	Primary Key	Foreign Keys	Other Attributes
Product	`product_id`	`supplier_id`	*name, price*
Supplier	`supplier_id`	—	*name, phone*
Customer	`customer_id`	—	*name, phone*
Inventory	`inventory_id`	`product_id`	*quantity*
Sale	`sale_id`	`product_id, customer_id`	*quantity_sold, sale_date*

No partial dependencies exist. 2NF is satisfied.

Step 3 — Third Normal Form (3NF)

Rule: Must be in 2NF, and no non-key attribute may transitively depend on the primary key through another non-key attribute.

We check every relation for transitive dependencies of the form **PK** → **X** → **Y** where X is not a key.

Analysis of each relation:

Product	`product_id → supplier_id → (supplier.name, supplier.phone)`	*supplier_id is a FK, not a non-key attribute causing a transitive chain within Product. supplier details live in a separate Supplier table. OK.*
Supplier	`supplier_id → name, phone`	*All attributes depend directly on the PK. No transitive dependency. OK.*
Customer	`customer_id → name, phone`	*All attributes depend directly on the PK. No transitive dependency. OK.*
Inventory	`inventory_id → product_id → quantity (via inventory_id directly)`	*quantity depends on inventory_id, not transitively through product_id. OK.*
Sale	`sale_id → product_id, customer_id, quantity_sold, sale_date`	*All non-key attributes depend directly on sale_id. No transitive dependency. OK.*

No transitive dependencies exist in any relation. 3NF is satisfied.

Step 4 — Boyce-Codd Normal Form (BCNF)

Rule: For every non-trivial functional dependency **X** → **Y**, X must be a **superkey** (candidate key or superset of one). BCNF is stricter than 3NF and catches anomalies that 3NF misses.

Candidate Key analysis:

Relation	Candidate Key(s)	All Determinants Are Superkeys?
Product	`product_id`	**Yes**
Supplier	`supplier_id`	**Yes**
Customer	`customer_id`	**Yes**
Inventory	`inventory_id`	**Yes**
Sale	`sale_id`	**Yes**

In each relation, the only determinants in non-trivial FDs are the primary keys, which are candidate keys. There are **no BCNF violations**.

Complete set of functional dependencies (final schema):

Determinant	→	Dependent
`product_id`	→	`name, price, supplier_id`
`supplier_id`	→	`name, phone`
`customer_id`	→	`name, phone`
`inventory_id`	→	`product_id, quantity`
`sale_id`	→	`product_id, customer_id, quantity_sold, sale_date`

All determinants are candidate keys. BCNF is satisfied.

Final Normalized Schema

The final schema after achieving BCNF. All tables are free from redundancy, partial dependencies, transitive dependencies, and BCNF violations.

SUPPLIER

Column	Type	Key	Description
supplier_id	*INT*	PK	Unique supplier identifier
name	*VARCHAR(100)*		Supplier company name
phone	*VARCHAR(20)*		Contact phone number

PRODUCT

Column	Type	Key	Description
product_id	*INT*	PK	Unique product identifier
supplier_id	*INT*	FK	References SUPPLIER
name	*VARCHAR(100)*		Product name
price	*DECIMAL(10,2)*		Unit selling price

CUSTOMER

Column	Type	Key	Description
customer_id	*INT*	PK	Unique customer identifier
name	*VARCHAR(100)*		Customer full name
phone	*VARCHAR(20)*		Contact phone number

INVENTORY

Column	Type	Key	Description
inventory_id	*INT*	PK	Unique inventory record
product_id	*INT*	FK	References PRODUCT
quantity	*INT*		Current stock quantity

SALE

Column	Type	Key	Description
`sale_id`	*INT*	**PK**	Unique sale transaction
`product_id`	*INT*	**FK**	References PRODUCT
`customer_id`	*INT*	**FK**	References CUSTOMER
`quantity_sold`	*INT*		Units sold in this transaction
`sale_date`	*DATE*		Date of transaction

Summary

Normal Form	Violation Check	Result
1NF	Atomic values, single-valued columns, each row uniquely identified	**PASS**
2NF	No partial dependency on composite primary key	**PASS**
3NF	No transitive dependency through a non-key attribute	**PASS**
BCNF	Every determinant in every FD is a candidate key	**PASS**

The Inventory and Sales Management System database is fully normalized to BCNF. The design eliminates all forms of redundancy and anomalies while preserving all functional dependencies across five clean, well-structured relations.